

Intelligence artificielle

Programmation des jeux de réflexion

Elise Bonzon

elise.bonzon@u-paris.fr

- L'adversaire est imprévisible $\Rightarrow$ la solution doit le prendre en compte
- Un temps limite est imposé $\Rightarrow$ quand il n'est pas possible d'atteindre le but il faut être capable de l'approximer
- Pistes étudiées :
 - algorithme pour joueur parfait (Von Neumann, 1944)
 - horizon fini, évaluation approximative (Zuse, 1945 ; Shannon 1950)
 - élagage pour réduire le coût de recherche (McCarthy, 1956)

Différents types de jeux

Information	Déterministe	Hasard
Parfaite	échecs, reversi, go, ...	backgammon, monopoli, ...
Imparfaite	bataille navale, mastermind, ...	bridge, poker, scrabble, ...

Définition d'un jeu

Un jeu peut être formellement défini comme un problème de recherche, avec :

- S_0 : l'état initial, qui spécifie l'état du jeu au début de la partie
- $\text{Player}(s)$: définit quel joueur doit jouer dans l'état s
- $\text{Action}(s)$: retourne l'ensemble d'actions possibles dans l'état s
- $\text{Result}(s, a)$: fonction de transition, qui définit quel est le résultat de l'action a dans un état s
- $\text{TerminalTest}(s)$: test de terminaison. Vrai si le jeu est fini dans l'état s , faux sinon. Les états dans lesquels le jeu est terminé sont appelés états terminaux.
- $\text{Utility}(s, p)$: une fonction d'utilité associe une valeur numérique à chaque état terminal s pour un joueur p

Jeux à somme nulle

Un **jeu à somme nulle** est un jeu pour lequel la somme des utilités de tous les joueurs est la même pour toutes les issues possible du jeu

- Le nom peut prêter à confusion, **jeu à somme constante** serait plus approprié
- Par exemple, le jeu d'échecs est un jeu à somme nulle :
 - $0 + 1$
 - $1 + 0$
 - $\frac{1}{2} + \frac{1}{2}$

1. L'algorithme Minimax
2. L'élagage α - β
3. Les jeux déterministes en pratique
4. Jeux non-déterministes
5. Jeux à information imparfaite
6. Conclusion

L'algorithme Minimax

L'algorithme Minimax

L'algorithme Minimax s'applique sur des jeux :

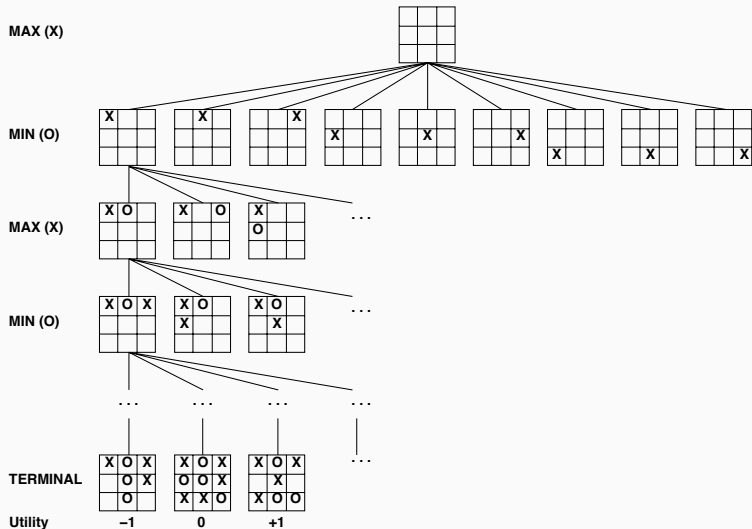
- à deux joueurs
- à somme nulle (ou somme constante)

Représentation arborescente

On peut représenter une partie à l'aide d'un arbre :

- A chaque niveau de l'arbre, un joueur choisit un coup
- Chaque joueur cherche à gagner
 - Le jeu est à somme constante : ce que l'un gagne, l'autre perd
 - Un joueur cherche à maximiser son score
 - Son adversaire cherche à le faire perdre
 - Cherche à minimiser le score (pour maximiser ses gains)
- On se place du point de vue du joueur qui commence à jouer
 - On appelle ce joueur MAX (il cherche à maximiser son score)
 - L'autre joueur est appelé MIN (il cherche à minimiser le score de MAX)

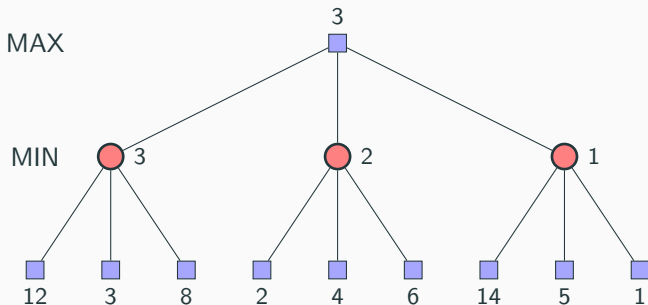
Arbre de recherche du *tic-tac-toe*



Algorithme de recherche Minimax

Minimax : Donne le **coup parfait** pour un jeu **déterministe à information parfaite à somme nulle**

→ choisir le coup qui mène vers l'état qui a la meilleure **valeur minimax**
= meilleure valeur possible contre le meilleur jeu de l'adversaire



Algorithme de recherche Minimax

```
1  function MINIMAX_DECISION(s) returns an action
2      for each a  $\in$  Action(s) do
3          Value(a)  $\leftarrow$  MIN_VALUE(result(a, s))
4      return the a with the highest Value(a)
```

```
1  function MIN_VALUE(s) returns a utility value
2      if TerminalTest(s) then return Utility(s)
3      v  $\leftarrow \infty$ 
4      for each a  $\in$  Action(s) do
5          v  $\leftarrow$  min{v, MAX_VALUE(result(a,s))}
6      return v
```

```
1  function MAX_VALUE(s) returns a utility value
2      if TerminalTest(s) then return Utility(s)
3      v  $\leftarrow -\infty$ 
4      for each a  $\in$  Action(s) do
5          v  $\leftarrow$  max{v, MIN_VALUE(result(a,s))}
6      return v
```

Et avec plus de deux joueurs ?

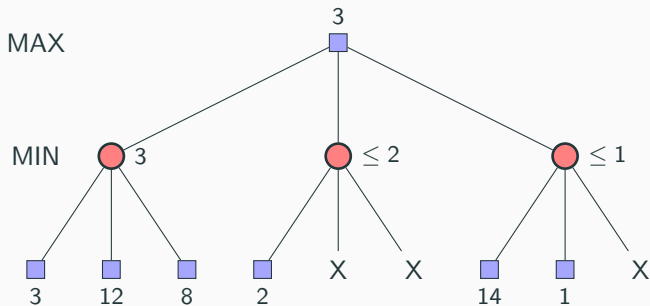
- L'algorithme Minimax peut marcher avec plus de deux joueurs
- Mais les autres joueurs n'ont pas toujours le même but (de minimiser le score du même joueur)
- Il faut donc analyser plus finement le jeu
 - Théorie des jeux (économie...)

Algorithme de recherche Minimax

- Complet, si l'arbre est fini
- Optimal **si l'adversaire est optimal**
- Si b est le nombre maximal de coups possibles et m la profondeur maximale de l'arbre
 - Complexité en temps : $O(b^m)$
 - Complexité en espace : $O(bm)$
- Pour les échecs par exemple : $b \sim 35$, $m \sim 100 \Rightarrow$ solution exacte impossible
- Mais avons nous besoin d'explorer tous les chemins ?

L'élagage α - β

Exemple d'élagage α - β



Elagage α - β ?

L'idée est donc de maintenir deux valeurs :

- α : valeur de la meilleure (plus haute) valeur jusqu'ici pour MAX
- β : valeur de la meilleure (plus basse) valeur jusqu'ici pour MIN

Le but est donc de mettre à jour α et β et d'élaguer la recherche quand la valeur du noeud considéré est pire que α pour MAX ou β pour MIN.

Algorithm α - β

```
1  function ALPHA_BETA(s) returns an action
2      v  $\leftarrow$  MAX_VALUE(s,  $-\infty$ ,  $+\infty$ )
3      return a  $\in$  Action(s) with value v
```

```
1  function MAX_VALUE(s,  $\alpha$ ,  $\beta$ ) returns a utility value
2      if TerminalTest(s) then return Utility(s)
3      v  $\leftarrow$   $-\infty$ 
4      for each a in Action(s) do
5          v  $\leftarrow$  max{v, MIN_VALUE(result(a,s),  $\alpha$ ,  $\beta$ )}
6          if v  $\geq$   $\beta$  then return v
7           $\alpha$   $\leftarrow$  max{ $\alpha$ , v}
8      return v
```

```
1  function MIN_VALUE(s,  $\alpha$ ,  $\beta$ ) returns a utility value
2      if TerminalTest(s) then return Utility(s)
3      v  $\leftarrow$   $+\infty$ 
4      for each a in Action(s) do
5          v  $\leftarrow$  min{v, MAX_VALUE(result(a,s),  $\alpha$ ,  $\beta$ )}
6          if v  $\leq$   $\alpha$  then return v
7           $\beta$   $\leftarrow$  min{ $\beta$ , v}
8      return v
```

- L'élagage n'affecte pas le résultat final
- Un bon choix améliore l'efficacité de l'élagage
- Avec un “choix parfait” la complexité en temps est $O(b^{m/2})$
 - ⇒ double la profondeur de recherche par rapport à Minimax
 - ⇒ Mais trouver la solution exacte avec une complexité de l'ordre de 35^{50} , pour les échecs, est toujours impossible

Approches standard en cas de ressources limitées, ou d'espace de recherche trop grand :

- Utiliser un test d'arrêt : remplacer `TerminalTest` par `CutoffTest`
 - pour, par exemple, limiter la profondeur
- Utiliser une fonction d'évaluation : remplacer `Utility` par `Eval`
 - valeur estimée d'une position

Par exemple, si on peut explorer 10^4 nœuds par seconde et que l'on a 100 secondes

⇒ On peut regarder 10^6 nœuds par coup $\sim 35^{8/2}$

⇒ $\alpha\text{-}\beta$ peut facilement atteindre des profondeurs de l'ordre de 8 coups d'avance aux échecs

⇒ plutôt bon programme d'échecs

Fonctions d'évaluation

Nous avons donc besoin d'évaluer un état du jeu **avant qu'il soit fini**.

Fonction d'évaluation

Une **fonction d'évaluation** estime la valeur d'une position.

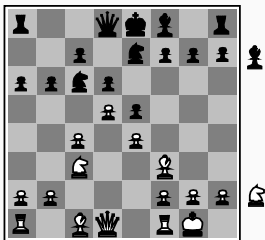
- $\text{Eval}(s) > 0$: position favorable
- $\text{Eval}(s) < 0$: position défavorable
- $\text{Eval}(s) = 0$: position équilibrée
- $\text{Eval}(s) = +\infty$: position gagnante pour le joueur courant
- $\text{Eval}(s) = -\infty$: position perdante pour le joueur courant

Par exemple, pour le *tic-tac-toe* :

$$\text{Eval}(s) = (\#3\text{cases pour moi}) - (\#3\text{cases pour mon adversaire})$$

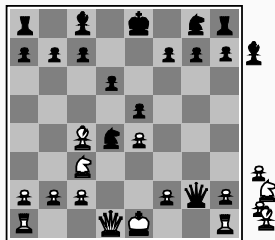
3cases : ligne(s), colonne(s) ou diagonale(s) pour lesquelles il est possible d'aligner 3 pions

Fonction d'évaluation : les échecs



Black to move

White slightly better

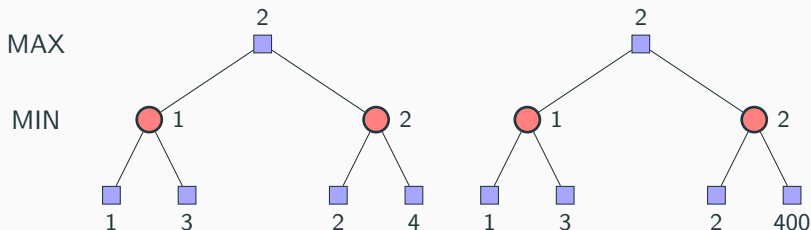


White to move

Black winning

- Aux échecs, on choisit par exemple une somme linéaire pondérée de caractéristiques
- $\text{Eval}(s) = w_1 f_1(s) + w_2 f_2(s) + \dots + w_n f_n(s)$
- avec par exemple
 - $w_1 = 9$ et $f_1(s) = (\text{nombre de reines blanches}) - (\text{nombre de reines noires})$,
 - etc.

α - β : la valeur exacte des nœuds *n'est pas importante*



- Seul l'ordre dans lequel on visite les nœuds est important
- Le comportement est préservé pour chaque transformation **monotone** de la fonction **Eval**
- Pour les jeux déterministes, la fonction de résultat se comporte comme une **fonction d'utilité ordinale**

- L'ordre dans lequel on visite les fils est important
- Si on trouve rapidement une bonne valeur, on élague plus de nœuds
- On peut trier les fils par leur utilité
- D'autres améliorations sont possible

Aux échecs on utilise au début du jeu des bases de données d'ouverture, au milieu α - β , et à la fin des algorithmes spéciaux

Les jeux déterministes en pratique

Quelques ordres de grandeur

- Tic-tac-toe : 3^9 états
- Rubik's cube : 10^{19} états
- Dames : 10^{40} états
- Echecs : 10^{120} états (facteur de branchement : 35)
- Go : 10^{172} états (facteur de branchement : > 300)

- **Puissance 4** : Le jeu est résolu : on a montré qu'il existait une stratégie gagnante pour le joueur qui commence à jouer. Cette stratégie étant stockée dans une base de données, il est facile pour un joueur artificiel de suivre cette stratégie et de gagner systématiquement.
- **Dames** : Chinook a mis fin à un règne de 40 ans du champion du monde Marion Tinslay en 1994. Il a utilisé une base de données définissant un jeu parfait pour toutes les positions avec 8 pièces ou moins sur le damier. (Soit au total 443 748 401 247 positions)

- **Othello** : Les champions humains refusent de jouer contre les ordinateurs, qui sont trop bons
- **Echecs** : Deep Blue a battu le champion du monde Gary Kasparov dans un match à 6 parties en 1997. Deep Blue cherche 200 million de positions par seconde, utilise une fonction d'évaluation très sophistiquée, et des méthodes non communiquées pour étendre quelques lignes de recherche jusqu'à 40 coups d'avance. En 2017, Stockfish est le meilleur logiciel d'échecs. Son niveau est largement supérieur à celui des meilleurs joueurs humains.

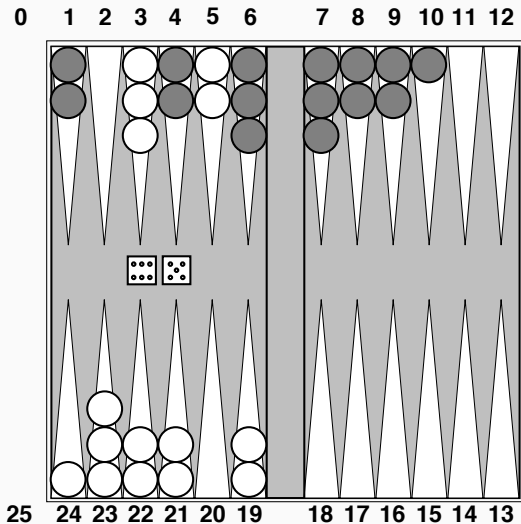
Les jeux déterministes en pratique

- **Go :**

- $b > 300$, dans les années 90 la plupart des programmes utilisaient des bases de connaissances permettant de suggérer les coups les plus probables.
- 2006, Monte-Carlo Tree Search (MCTS).
- 2016, AlphaGo (Google Deepmind) a battu un joueur humain professionnel. AlphaGo utilise MCTS, des simulations et un réseau de neurones profond pour orienter la recherche. Le réseau de neurones a appris en utilisant des bases de coups joués par des joueurs humains professionnels.
- 2017, AlphaGo Zéro confirme en battant un autre champion humain. Le réseau de neurones a appris en auto-jeu, sans utiliser de connaissances humaines.
- Alpha Zéro apprend à jouer aux Echecs et bat Stockfish ; apprend à jouer au Shogi et bat le meilleur logiciel de Shogi. Approche générale pour les jeux déterministes à information complète.

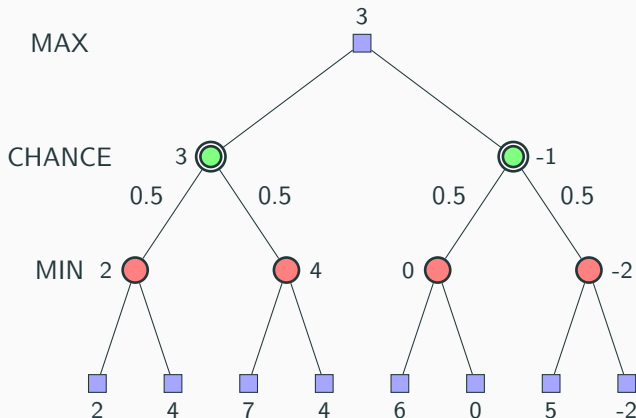
Jeux non-déterministes

Jeu non-déterministe : backgammon



Jeux non-déterministes

- Dans un jeu non-déterministe, la chance est introduite par un jet de dés, une distribution de cartes ...
- Exemple simplifié avec le lancer d'une pièce de monnaie :



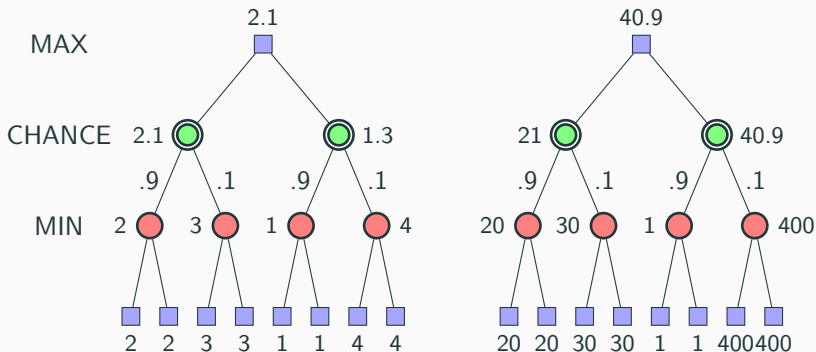
Algorithme pour les jeux non-déterministes

- Expectiminimax donne le coup parfait, comme MiniMax
- On doit considérer en plus les nœuds Chance
- On doit ajouter la ligne suivante à l'algorithme MiniMax ::

```
1  if Player(s) = Max then
2      return the highest Expectiminimax-Value of Result(s,a)
3  if Player(s) = Min then
4      return the lowest Expectiminimax-Value of Result(s,a)
5  if Player(s) = Chance then
6      return the average Expectiminimax-Value of Result(s,a)
7
```

- Si Eval est bornée, α - β peut-être adaptée

La valeur exacte des nœuds *est* importante



- Le comportement est préservé seulement pour chaque transformation **positive et linéaire** de la fonction **Eval**
- **Eval** doit être proportionnelle au gain attendu

Jeux à information imparfaite

Jeux à information imparfaite

- Par exemple, jeux de cartes où les cartes de l'adversaire ne sont pas connues
- Il est possible de calculer une probabilité pour chaque distribution de cartes possible
- Ressemble à un *gros dé* lancé au début du jeu
- **Idée** : calculer la valeur Minimax de chaque action dans chaque distribution possible, puis choisir l'action qui la valeur espérée maximale parmi toutes les distributions
- Cas spécial : si une action est optimale pour toutes les distributions, c'est une action optimale

Exemple de sens commun

- Jour 1
 - Route A mène à un petit tas de pièces d'or
 - Route B mène à une intersection
 - Si vous prenez à gauche, vous arriverez à une montagne de bijoux
 - Si vous prenez à droite, vous vous ferez écraser par un bus
- Jour 2
 - Route A mène à un petit tas de pièces d'or
 - Route B mène à une intersection
 - Si vous prenez à gauche, vous vous ferez écraser par un bus
 - Si vous prenez à droite, vous arriverez à une montagne de bijoux
- Jour 3
 - Route A mène à un petit tas de pièces d'or
 - Route B mène à une intersection
 - Devinez correctement et vous arriverez à une montagne de bijoux
 - Devinez incorrectement et vous vous ferez écraser par un bus

Jeux à information imparfaite

- L'idée que la valeur d'une action est la moyenne de ses valeurs possibles est **fausse**
- Avec une observation partielle, la valeur d'une action dépend de **l'état des connaissances** ou **l'état de croyance** de l'agent
- Il est possible de générer un arbre de recherche sur les états de connaissances
- Mène à des comportements rationnels tels que :
 - Agir pour obtenir de l'information
 - Donner les informations que l'on a à son ou ses partenaire(s)
 - Agir au hasard pour minimiser les informations que l'on fournit à ses adversaires

Conclusion

- Etudier les jeux permet de soulever plusieurs points importants en IA
 - La perfection n'est pas atteignable $\Rightarrow$ nécessité d'approximer
 - Il faut penser à quoi penser
 - L'incertitude force à assigner des valeurs aux états
 - Les décisions optimales dépendent des informations sur l'état, pas de l'état réel
- Les jeux sont à l'IA ce que les grands prix sont à la conception des voitures

Et le projet dans tout ça ?

- Choisir un jeu assez difficile pour que ce soit intéressant
 - Le morpion, avec 3^9 états, peut être résolu avec un simple Minimax
- Mais pas trop compliqué non plus...
 - Les règles du jeu d'échec sont très complexes à programmer !
- Quelques exemples de jeux possibles (liste non exhaustive) : Dames (et ses variantes) (mais il y en a souvent beaucoup) ; Gomoku ; Othello ; Puissance 4 (mais il y en a souvent beaucoup aussi) ; Quarto ; Quoridor ; Abalone ; Jeu du Hex ; Jeu du moulin ; Otrio ; Squadro ; ...

L'originalité du jeu choisi ainsi que sa difficulté intrinsèque (en particulier au regard du facteur de branchement et de la profondeur de recherche induite) seront prises en compte dans le barème d'évaluation.

Méthodologie proposée

1. Définir la structure représentant les états (données + fonctions)
2. Implémenter la boucle de jeu pour 2 joueurs humains
3. Implémenter l'algorithme Minimax basique
4. Remplacer un joueur humain par une IA avec l'algorithme Minimax
5. Implémenter et intégrer l'élagage $\alpha\beta$
6. Implémenter plusieurs difficultés de jeu (au moins 3)
 - Plusieurs profondeurs de recherche
 - Plusieurs fonctions d'évaluation
7. Faire un tournoi de vos IAs
 - Tester vos différentes IA les unes contre les autres pour vérifier qu'elles ont bien la difficulté escomptée
 - Au moins 50 tests par couple d'IA (plus, ça peut être bien)

1. Présentation générale du jeu
2. Implémentation et validation du moteur de jeu
 - choix d'implémentation du moteur de jeu
 - **tests unitaires** pour vérifier la correction des règles du jeu
 - Code compilable et exécutable sur une machine Linux
3. Fonctions d'évaluation, score et élagage alpha-bêta
 - Description des fonctions d'évaluation
 - Analyse expérimentale des fonctions d'évaluation
4. Tournoi entre les intelligences artificielles
5. Utilisation de l'IA générative
 - Quels modèles ?
 - Analyse critique de l'apport et des limites de ces outils dans le cadre du projet
6. Difficultés rencontrées

- **Rapport** et **code** à rendre **avant le 10/05/2026**
 - Un seul fichier **.zip** par binôme
 - Le **code**, documenté et commenté
 - Un **makefile** ou un **script shell** pour produire un exécutable ou un fichier jar (si langage C, C++, Java)
 - Un fichier **README**
 - Le **rapport**, en **pdf**
- **Soutenances** fin mai, après les examens, en fonction du programme des soutenances des projets tuteurés
 - Démonstration du jeu
 - Questions sur le code